example to understand the operations of Bloom filter.

- For an input set, S = *[1,2,3,4]*
- Number of hash functions, *k=2* ;
- Size of Bloom Filter, *m=8* ;
- We have two hash functions: h_1,h_2

Bloom filter preparation

The preparation phase involves setting an entry using the *add* operation in the Bloom filter corresponding to a set member. The *add* operation maps the input element using the two hash functions. Each hash function output is an index into the Bloom filter array *B*. After calculating the index, it is set to one. This process will be repeated for all the input entries. The *add* operation is defined as follows:

```
add(x):
    i = h1(x)
    j = h2(x)
    B[i] = 1
    B[j] = 1
```

The Bloom filter bit array is as follows:

0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0

```
add (2)
h1(2) = 6
h2(2) = 7
```

0	1	2	3	4	5	6	7
0	0	0	0	0	0	1	1

```
add (3)
h1(3) = 4
h2(3) = 1
```

0	1	2	3	4	5	6	7
0	1	0	0	1	0	1	1

To determine the membership of an element in an input set S, the *check* operation is used. This operation is defined as follows:

```
bool check(x):
    i = h1(x)
    j = h2(x)
    return i && j
```

The input value is subjected to the same hash functions, and the corresponding bits at the indices are checked if they are set. If all the values are set, then the input value might be present in the set. This is always probable, but not certain because the bits on the positions in question might be set by other *add* operations of other elements.

When the filter returns *False* for a given input, it is certain that the input element does not exist in the set. The condition of a false return will occur when one or more hashed indices have zero value. No *add* operation ever sets those positions to be 1, and hence the input element is not present in the set.

```
check (7)
h1(3) = 4
h2(3) = 0
```

0	1	2	3	4	5	6	7
0	1	0	0	1	0	1	1

In the above operation, since a check would return *False*, the element is definitely not present in the set. This is the utility of the Bloom filter, which eliminates spurious lookup operations.

```
check (5)
h1(3) = 6
h2(3) = 7
```

0	1	2	3	4	5	6	7
0	1	0	0	1	0	1	1

In the operation given above, since the check returns *True*, it is possible that 5 is present in the set. This is a false positive result and an error of the filter.

The error rate of a Bloom filter

The expected value of false positive probability *p* decides the optimal values for Bloom filter size *m*, and the number of hash functions *k*, for an input list of elements. We assume that all the *k* hash functions are uniform and will choose all *m* positions in the bit array *B* of Bloom Filter with equal probability of $\frac{1}{m}$.

The probability of choosing a position among *m* positions is $\frac{1}{m}$. The probability of not choosing a position is $(1-(\frac{1}{m}))$. Since there are *k* hash functions, the probability of not choosing that position for a position is $(1-(\frac{1}{m}))^k$.

For *n* entries, we repeat the above process *n* times and get the probability of not setting that position as $(1-(\frac{1}{m}))^{kn}$. So, the probability that the position is set after the above process is $(1-(1(\frac{1}{m}))^{kn})$. Considering all the positions that are set, the final probability we get is $(1-(1(\frac{1}{m}))^{kn})^k$ which is approximately $(1-e^{-(\frac{kn}{m})})^k$.

The effectiveness of the Bloom filter depends on the values of *p*, *k*, *n* and *m*. We need to find optimal values of *m* and *k* depending on the input values of *p* and *n*. By the use of differential calculus, we get the following results, for